Logging Using SLF4J

Required Libraries

SLF4J provides the logging API and Logback provides the logging implementation. These libraries are used for all three exercises.

```
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>1.7.30</version>
</dependency>
<dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.2.3</version>
</dependency>
```

Exercise 1: Logging Error Messages and Warning Levels

Task

Write a Java application that demonstrates logging error messages and warning levels using SLF4J.

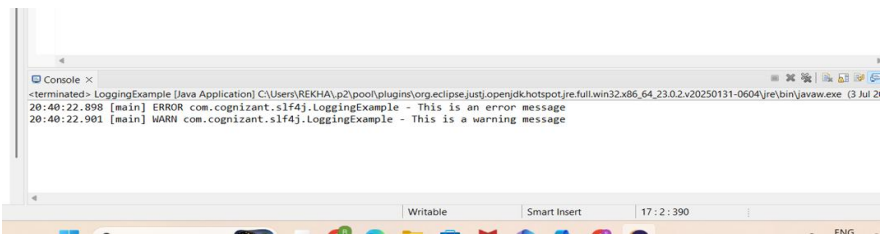
Java Code

```
package com.cognizant.slf4j;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {
    private static final Logger logger =
            LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {
        logger.error("This is an error message");
        logger.warn("This is a warning message");
    }
}
```

Explanation

The Logger is created using LoggerFactory. The error() method records an ERROR-level message and warn() records a WARN-level message. The console output confirms successful execution.

Output



Exercise 2: Parameterized Logging

Task

Write a Java application that demonstrates parameterized logging using SLF4J.

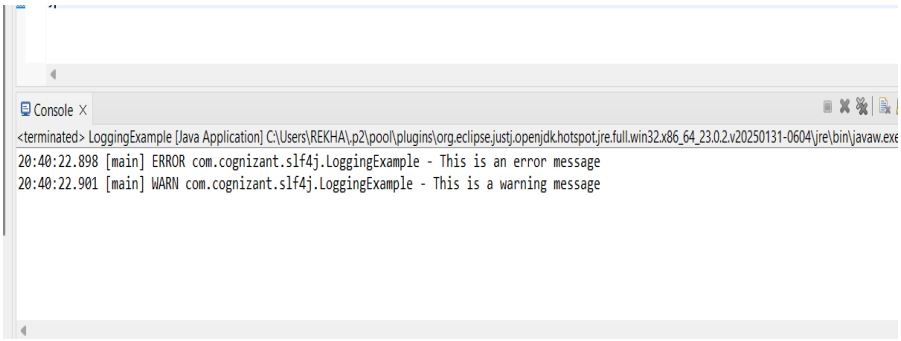
Java Code

```
package com.cognizant.slf4j;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class ParameterizedLoggingExample {
    private static final Logger logger =
            LoggerFactory.getLogger(ParameterizedLoggingExample.class);

    public static void main(String[] args) {
        String user = "Rekha";
        int age = 21;

        logger.info("User {} is {} years old", user, age);
        logger.warn("User {} has attempted login {} times", user, 3);
    }
}
```

Explanation

Parameterized logging uses {} placeholders. SLF4J replaces them with supplied values at runtime, making the code cleaner and avoiding manual string concatenation.

Output



Exercise 3: Using Different Appenders

Task

Write a Java application that demonstrates console and file appenders with SLF4J and Logback.

logback.xml Configuration

```
<configuration>
    <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>
    <appender name="file" class="ch.qos.logback.core.FileAppender">
        <file>app.log</file>
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>
    <root level="debug">
        <appender-ref ref="console"/>
        <appender-ref ref="file"/>
    </root>
</configuration>
```

Java Code

```
package com.cognizant.slf4j;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AppenderExample {
    private static final Logger logger =
            LoggerFactory.getLogger(AppenderExample.class);

    public static void main(String[] args) {
        logger.debug("This is a debug message");
        logger.info("This is an information message");
        logger.warn("This is a warning message");
        logger.error("This is an error message");
    }
}
```

Explanation

The ConsoleAppender displays logs in Eclipse and the FileAppender saves the same logs in app.log. With the root level set to debug, DEBUG, INFO, WARN, and ERROR messages are processed.

Output